

CSE464 Course Project – Part 3

Overview

In this project, I improved my graph search program by making the code more organized and easier to understand. I refactored the code and added new features like different search algorithms and a random walk search.

GitHub Link: <https://github.com/speedylight61-del/CSE464-2026-frgibbs.git>

How to Run the Program

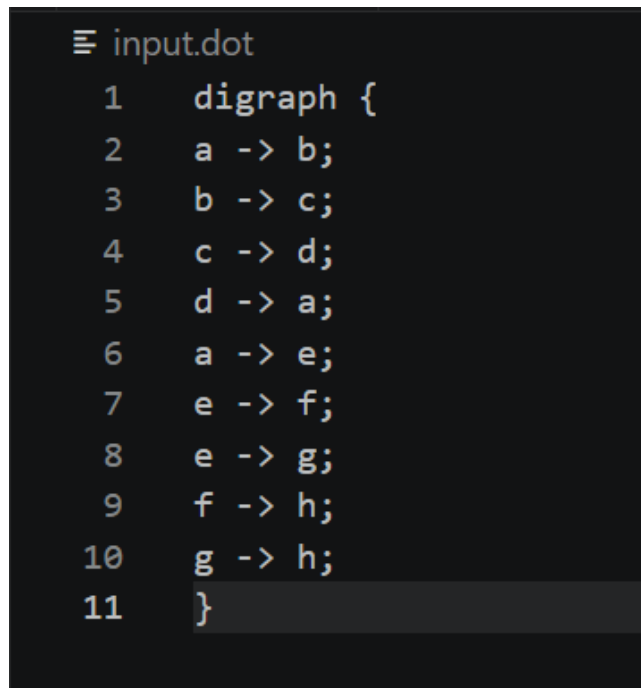
1. Open the project in your IDE (VS Code or IntelliJ)
2. Make sure Maven is installed
3. Open the terminal in the project folder
4. Run this command:

```
mvn exec:java "-Dexec.mainClass=Main"
```

Input File

The program uses a .dot file as input.

```
digraph {  
a -> b;  
b -> c;  
c -> d;  
d -> a;  
a -> e;  
e -> f;  
e -> g;  
f -> h;  
g -> h;  
}
```

A screenshot of a code editor with a dark background. The file name 'input.dot' is shown in the top left corner. The code is a directed graph definition in DOT format, with line numbers 1 through 11 on the left margin. The code defines a digraph with nodes a, b, c, d, e, f, g, and h, and directed edges between them.

```
1  digraph {  
2    a -> b;  
3    b -> c;  
4    c -> d;  
5    d -> a;  
6    a -> e;  
7    e -> f;  
8    e -> g;  
9    f -> h;  
10   g -> h;  
11   }
```

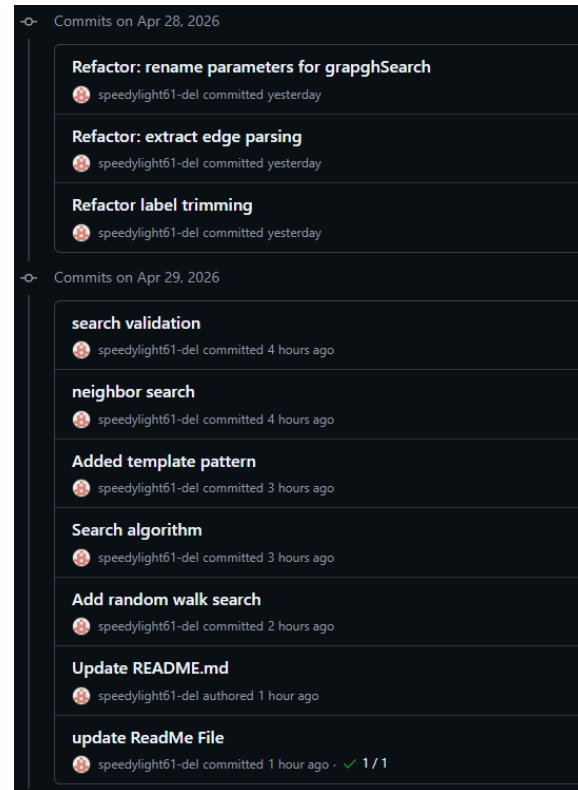
1. Refactoring (Q1)

In this part, I improved my graph search code to make it easier to read and understand. I made several small changes to clean up the code and remove repeated logic.

Some of the changes I made include:

- improving the names of variables in the graphSearch method
- moving repeated code into helper methods
- cleaning up labels using a single method
- adding checks to make sure the input is valid
- making neighbor lookup clearer

These changes helped make the code more organized and easier to maintain.



2. Template Pattern (Q2)

I used the Template Method Pattern to reduce repeated code between BFS and DFS. I created a base class called AbstractGraphSearch that contains the main steps of the search process. This includes checking the nodes, visiting nodes, and building the final path.

Then I created two classes, BfsSearch and DfsSearch, that extend the base class.

These classes only change how nodes are processed, while the main structure stays the same. This works because both BFS and DFS follow similar steps, but they explore the graph differently. Using this pattern made my code cleaner and avoided repeating the same logic.

```
1 import java.util.*;
2
3 public abstract class AbstractGraphSearch implements Search {
4
5     public String search(Graph graph, String source, String destination) {
6         if (source == null || destination == null) return null;
7         source = source.trim();
8         destination = destination.trim();
9         if (!graph.getNodes().contains(source) || !graph.getNodes().contains(destination)) {
10             return null;
11         }
12         Set<String> visited = new LinkedHashSet<>();
13         Map<String, String> parent = new HashMap<>();
14         setup(source);
15         visited.add(source);
16         while (!isEmpty()) {
17             String current = nextNode();
18             if (current.equals(destination)) {
19                 return buildPath(destination, parent);
20             }
21             for (String neighbor : graph.getNeighbors(current)) {
22                 if (!visited.contains(neighbor)) {
23                     visited.add(neighbor);
24                     parent.put(neighbor, current);
25                     addNode(neighbor);
26                 }
27             }
28         }
29         return null;
30     }
31     protected abstract void setup(String source);
32     protected abstract boolean isEmpty();
33     protected abstract void addNode(String node);
34     protected abstract String nextNode();
35
36     protected String buildPath(String destination, Map<String, String> parent) {
37         String path = destination;
38         String cur = destination;
39         while (parent.containsKey(cur)) {
40             cur = parent.get(cur);
41             path = cur + " -> " + path;
42         }
43         return path;
44     }
45 }
```

3. Strategy Pattern (Q3)

I used the Strategy Pattern to allow the program to choose which search algorithm to run.

I created an interface called Search, and then made three classes that use it:

- BfsSearch
- DfsSearch
- RandomWalkSearch

In the graphSearch method, the program checks which algorithm is selected and runs the correct one. This makes it easy to switch between different search methods without changing the main code. This pattern makes the program more flexible and easier to expand in the future.

```
1 public interface Search {  
2     String search(Graph graph, String source, String destination);  
3 }
```

```
public String graphSearch(String source, String destination, Algorithm algo) {  
    Search strategy;  
    if (algo == Algorithm.BFS) {  
        strategy = new BfsSearch();  
    } else if (algo == Algorithm.DFS) {  
        strategy = new DfsSearch();  
    } else {  
        strategy = new RandomWalkSearch();  
    }  
    return strategy.search(this, source, destination);  
}
```

4. Random Walk Search (Q4)

I added a new search algorithm called Random Walk. This algorithm starts at the source node and randomly chooses the next node from its neighbors. Because it is random, the path can be different each time the program runs.

I tested the program multiple times and saw different paths in the output. This shows that the algorithm is working correctly. Each time I ran the program, the path changed, like sometimes it went from $a \rightarrow b \rightarrow c$, and other times it went from $a \rightarrow e \rightarrow f \rightarrow h$. This shows that the algorithm is random and working correctly.

```
Graph saved to graphOutput.txt

--- BFS Search ---
Visit Node History: a
Visit Node History: a-b
Visit Node History: a-e
Visit Node History: a-b-c
Visit Node History: a-e-f
Visit Node History: a-e-g
Visit Node History: a-b-c-d
Visit Node History: a-e-f-h
Found target node: h

--- DFS Search ---
Visit Node History: a
Visit Node History: a-e
Visit Node History: a-e-g
Visit Node History: a-e-g-h
Found target node: h

Number of edges: 9
Edges:
a -> b
b -> c
c -> d
d -> a
a -> e
e -> f
f -> g
g -> h

Graph saved to graphOutput.txt
random testing
Visit Node History: a
Visit Node History: a-b
Visit Node History: a-b-c
Found target node: c

random testing
Visit Node History: a
Visit Node History: a-b
Visit Node History: a-b-c
Found target node: c

random testing
Visit Node History: a
Visit Node History: a-b
Visit Node History: a-b-c
Found target node: c

random testing
Visit Node History: a
Visit Node History: a-e
Visit Node History: a-e-g
Visit Node History: a-e-g-h
Reached dead end at node h

[INFO] -----
[INFO] BUILD SUCCESS
```

5. Pull Request and Code Review (Q5)

I created a pull request on GitHub to show all the changes I made in this project. The pull request includes my refactoring work, the design patterns, and the Random Walk search. I also reviewed my code and added comments to explain what I improved. This helped me make sure my code was clear and correct.

Pull Request Link:

<https://github.com/speedylight61-del/CSE464-2026-frgibbs/pull/1>

